



CSI142 Object-Oriented Programming

Advanced Control Structures I

Week 2 • Lecture 4

switch • operator precedence • do-while

Arrays and ArrayList refresher • command-line arguments

Mon 2 Feb 2026 • 08:00–09:00 • Room 252-008



- **Use switch for discrete choices and avoid fall-through errors**
- **Use parentheses to make boolean logic unambiguous (operator precedence)**
- **Apply do-while for “prompt at least once” workflows**
- **Run programs with command-line arguments and guard `args.length`**



- **Q1: What is the difference between = and == ?**
- **Q2: Why can nextInt() followed by nextLine() be tricky?**
- **Q3: Give one example of a sentinel value.**



switch: when is it the right tool?

CSI142 • Semester II 2025/26

Good fit

- **Menu options: 1,2,3...**
- **Day-of-week logic**
- **Command words (String)**
- **Many discrete cases**

Not a good fit

- **Ranges: score ≥ 50**
- **Complex conditions**
- **Overlapping cases**
- **When if-else is clearer**



switch syntax (and why break matters)

CSI142 • Semester II 2025/26

```
1 switch (choice) {  
2     case 1:  
3         System.out.println("Add item");  
4         break;  
5     case 2:  
6         System.out.println("Print items");  
7         break;  
8     case 3:  
9         System.out.println("Exit");  
10        break;  
11    default:  
12        System.out.println("Invalid choice");  
13 }
```

Key points

- Use break unless you intentionally want fall-through
- default handles unexpected inputs
- Keep cases short, call methods later (Week 3)



Pitfall: accidental fall-through

CSI142 • Semester II 2025/26

```
1 switch (day) {  
2     case 1:  
3         System.out.println("Mon");  
4     case 2:  
5         System.out.println("Tue");  
6     default:  
7         System.out.println("Other");  
8 }
```

Key points

- Predict before running
- Fix: add break
- If you want fall-through, comment it clearly



- Java evaluates **&&** before **||**
- Write parentheses for humans, not for the compiler
- Example: **`a == b || c < d && e == f`**
- Readable: **`(a == b) || ((c < d) && (e == f))`**



while

- Checks condition first
- May run zero times
- Good for “repeat while valid”

do-while

- Runs body once before checking
- Perfect for menus and prompts
- Common in console apps



The menu pattern: do-while + switch

CSI142 • Semester II 2025/26

```
1 int choice;
2 do {
3     System.out.println("1) Add  2) Print  3) Exit");
4     choice = in.nextInt();
5     switch (choice) {
6         case 1: /* add */ break;
7         case 2: /* print */ break;
8         case 3: System.out.println("Bye"); break;
9         default: System.out.println("Invalid");
10    }
11 } while (choice != 3);
```

Key points

- Display menu
- Read choice
- switch handles option
- Loop until Exit option



Array (fixed size)

- `int[] a = new int[5];`
- Fast, simple indexing
- Use when size is known
- `a.length` is constant

ArrayList (dynamic)

- `ArrayList<Integer> list = new ArrayList<>();`
- Grows as needed
- Use when size is unknown
- `list.size()` changes



```
1 public static void main(String[] args) {
2     boolean teacherMode =
3         args.length > 0 && args[0].equalsIgnoreCase("teacher");
4
5     if (teacherMode) {
6         System.out.println("Teacher features enabled");
7     } else {
8         System.out.println("Student mode");
9     }
10 }
```

Key points: • args is a String[], • Guard args.length, • Use args to toggle features



- **Design a menu for a “Mini Grade Manager” (3 options + Exit).**
- **Write the do-while condition, what value exits?**
- **Which switch cases need break?**
- **Exit ticket: one switch pitfall and one do-while use case.**